

Nexus AI — MERN Stack Master Interview & Architecture Guide

Author: B Surya Prakash | **Project:** MERN AI Platform | **Stack:** React 18, Node.js, Express, MongoDB, Stripe

Executive Pitch: Nexus AI is a full-stack MERN AI assistant platform built with React 18, Vite, Tailwind CSS v4, Node.js, Express, and MongoDB. Key architectural highlights include stateless JWT authentication, subdocument chat storage, atomic credit consumption via MongoDB \$inc, Stripe Checkout webhooks with cryptographic signature verification, and custom Markdown table rendering.

1. System Architecture & Tech Stack

- **Frontend:** React 18 (Vite, Tailwind CSS v4, Context API, React Router v7, React Hot Toast, Moment.js).
- **Backend:** Node.js & Express REST API with JWT Auth, Bcrypt password hashing, Mongoose ODM.
- **Database:** MongoDB Atlas with User, Chat (embedded messages), and Transaction collections.
- **Payment & AI:** Stripe Checkout SDK with raw body webhook verification & dynamic AI integrations.

2. Core API Endpoints Reference

Method	Endpoint	Protection	Description
POST	/api/user/register	Public	Registers user & returns JWT
POST	/api/user/login	Public	Authenticates credentials & returns JWT
GET	/api/user/data	Protected	Fetches user profile & credits
GET	/api/chat/get	Protected	Retrieves chat history
POST	/api/message/text	Protected	Processes prompt, checks credits & returns AI reply
POST	/api/credit/purchase	Protected	Initializes Stripe Checkout session
POST	/api/stripe	Raw Body	Stripe webhook for automated credit top-up

3. Engineering Challenges & Critical Fixes

- **JavaScript Truthy Bug ('success': 'false'):** Backend previously returned string 'false'. In JS, Boolean('false') === true, triggering success blocks on error responses. Fixed backend to return boolean false and updated frontend to strictly validate data.success === true.
- **Authorization Header Robustness:** Replaced fragile req.headers.authorization.split(' ')[1] with defensive checks handling optional Bearer prefixes and missing headers safely.
- **Stripe Webhook Signature Verification:** Registered /api/stripe with express.raw({ type: 'application/json' }) before general JSON body parsers to preserve the unparsed Buffer required by stripe.webhooks.constructEvent.
- **Custom Markdown & Mobile Viewport:** Developed FormattedMarkdown component supporting overflow-x-auto horizontal scroll containers for Markdown tables and code snippets on narrow mobile viewports.

4. Master Interview Questions & Answers (Stun the Interviewer)

Q1: How does authentication work end-to-end in this project?

Authentication is stateless and powered by JSON Web Tokens (JWT). Upon login/register, backend verifies credentials using Bcrypt and signs a 30-day JWT containing user `_id`. The client stores this in `localStorage` and `Context` state, sending it as ``Authorization: Bearer `` on protected routes. The ``protect`` middleware intercepts requests, verifies JWT validity, and attaches the user document to ``req.user``.

Q2: How do you prevent race conditions during credit deduction when generating messages?

We verify user credits `> 0` prior to AI dispatch. After receiving the AI response, credits are decremented atomically using MongoDB's ``$inc`` operator (``User.updateOne({ _id: userId }, { $inc: { credits: -1 } })``). Atomic database operators eliminate race conditions common in read-modify-write patterns.

Q3: How do Stripe Webhooks guarantee idempotency and prevent duplicate credit awards?

We record a `Transaction` document in MongoDB with ``isPaid: false`` when checkout starts. When Stripe emits ``payment_intent.succeeded``, our webhook verifies the signature and queries ``Transaction.findOne({ _id: transactionId, isPaid: false })``. If already paid, processing halts immediately. If unpaid, credits increment via ``$inc`` and ``isPaid`` flips to ``true`` atomically.

Q4: Why use embedded subdocuments for messages inside the Chat schema?

Messages and chats are always queried and updated together. Embedding messages as subdocuments inside the `Chat` document eliminates expensive ``$lookup`` joins and delivers single-document read/write performance well within MongoDB's 16MB document size limit.

Q5: How do you handle raw body parsing for Stripe webhooks alongside `express.json()`?

Express executes middleware sequentially. Standard ``express.json()`` converts raw JSON into objects, destroying the raw byte stream needed for Stripe signature verification. We mount ``express.raw({ type: 'application/json' })`` specifically on the ``/api/stripe`` endpoint prior to standard JSON body parsers.

Q6: How does the frontend handle optimistic UI rendering during message generation?

Upon prompt submission, ``Chatbox.jsx`` appends the prompt to local React state immediately and displays an animated pulsing loading indicator (``Nexus AI is generating response...``). Once the backend completes, we sync chat state via ``fetchUsersChats()``. Errors clear the loader and trigger ``react-hot-toast`` notifications.

Q7: How did you optimize mobile viewport responsiveness for complex data like Markdown tables?

We implemented responsive max-widths (``max-w-[85%] sm:max-w-2xl``) and flex layouts. For Markdown tables, our custom ``FormattedMarkdown`` parser wraps `` elements inside ``overflow-x-auto`` container divs, enabling smooth horizontal scrolling on mobile viewports (320px–390px) without breaking page layouts.

Q8: What security practices are implemented for password storage and token handling?

Passwords are never stored plain-text; Mongoose pre-save hooks hash passwords using ``bcrypt.genSalt(10)``. JWT tokens are cryptographically signed, transmitted over HTTPS authorization headers, and stored in React Context memory with `localStorage` session fallback.

Q9: How does the application handle external AI API outages or timeouts?

The call to the external LLM API is isolated within a ``try-catch`` block. If the AI service fails or times out, a clean fallback message is appended to the chat session without crashing the node process or throwing unhandled server exceptions.

Q10: How would you scale this MERN stack application to 100,000 active concurrent users?

1) Shard MongoDB chats collection by ``userId`` and create compound indexes on ``chats.userId`` and ``users.email``. 2) Cache user profiles and credits in Redis. 3) Replace standard HTTP POST with WebSockets/SSE for real-time token streaming. 4) Use BullMQ message queues for async prompt processing. 5) Scale Express horizontally in Docker containers behind an NGINX load balancer.

Q11: What is the purpose of `react-hot-toast` and how is it integrated globally?

We mount a single `` at the root in ``App.jsx``. This enables non-blocking global notification calls (``toast.success()``, ``toast.error()``) from any component or Context action.

Q12: Why use React Context API over Redux or prop drilling for this project?

The `AppContextProvider` centralizes application state (`user`, `token`, `chats`, `selectedChat`, `theme`) across components without prop drilling or Redux boilerplate, providing clean lightweight state management ideal for this application scope.

Q13: How do pre-save hooks in Mongoose work during document updates?

Mongoose `pre('save')` hooks run prior to `.save()`. In our `User` schema, we check `if (!this.isModified('psw')) return;` to avoid re-hashing an existing hash during non-password updates (like credit modifications).

Q14: How are dark mode and light mode themes implemented and persisted?

Theme state is initialized from `localStorage.getItem('theme')`. A `useEffect` hook dynamically toggles the `dark` class on `document.documentElement` (`element`), utilizing Tailwind CSS class-based dark mode rules.

Q15: What architectural patterns did you follow when designing REST routes in Express?

We followed a modular Controller-Route-Middleware pattern. Routes are partitioned logically into dedicated routers (`userRouter`, `chatRouter`, `messageRouter`, `creditRouter`), auth concerns are isolated in `protect` middleware, and business logic is encapsulated within pure controller functions.